

AgentWorkShop: A Governed Supervisory Execution Layer for Industrial Agent Teams

Anonymous Authors

Abstract—Supervisory optimization above the regulatory control layer—tuning recipes, schedules, and setpoints against quality, throughput, energy, or discharge targets—is repeated in every plant, and so is the integration work around it. Each deployment re-establishes which actor may change which parameter, which limits apply to the current product and run, whether the device accepted the command, and whether the process improved. This paper presents AgentWorkShop, a governed supervisory execution layer that turns those repeated concerns into one path for agent teams. Team identities connect to semantic data-acquisition (DAQ) and write-control (DCW) nodes through explicit capabilities; line, product, recipe, and run context is resolved at execution time; and admission, approval, actuation, readback, observation, verdict, and compensation are recorded as separate evidence states instead of one success flag. Five protocol families, self-describing node and driver contracts, versioned production objects, plugin extension points, and a typed software development kit (SDK) form the integration surface, while process models, objectives, guards, and control policies remain external inputs. The evaluation exercises this single governed path through four questions: protocol closure and governance attribution, agent-team missions on a single node and on a multi-node line, a deterministic closed loop measured against an offline optimum, and concurrent process missions. The governed path intercepts every configured out-of-window probe without blocking an admissible write, both missions close their governed loops, and the process missions reach their archived predicates, while task completion, readback, and attainment stay separately recorded. The results support an inspectable, simulation-tier architecture for industrial supervisory execution.

Index Terms—Agent teams, industrial informatics, recipe optimization, write governance.

I. INTRODUCTION

INDUSTRIAL optimization occurs above the regulatory control layer: engineers adjust recipes, schedules, and setpoints to meet quality, throughput, energy, or discharge objectives while PLCs and local controllers retain fast regulation. The supervisory decision, however, is only one part of the intervention: a usable execution path must also establish who may change which parameter, which limits are active, whether the device accepted the command, and whether the process improved. Because these concerns are normally embedded in application code, they are rebuilt for every new process, protocol, or agent framework, and the rebuild is where duplicated drivers, ad-hoc permission checks, and logs that cannot be tied back to a decision accumulate.

These concerns recur across plants whose physics and objectives differ— injection molding coupling part weight to pressure, time, and temperature; wastewater treatment (WWTP) balancing effluent constraints against aeration and dosing cost; annealing trading material properties against line

speed; and BOPET (biaxially oriented polyethylene terephthalate) coordinating casting and stretching sections. All need an auditable bridge from a team-level task to heterogeneous acquisition and control nodes.

Large language model (LLM) agents are being studied for industrial diagnosis, engineering assistance, and PLC-program generation [1–4], including simulated controller and machine interaction [5, 6]. Yet orchestration frameworks and protocol connectivity do not by themselves answer who is authorized, which production limits apply, or how a write is linked to later evidence (Section II). Moreover, a task state such as COMPLETED can coexist with a missed process objective.

AGENTWORKSHOP addresses this execution gap through member–node bindings and shared line, product, recipe, and run context. Nodes expose engineering quantities rather than register addresses; bindings scope sanctioned tools; and intervention records retain the proposal, execution result, observation, and verdict. Fig. 1 contrasts this organization with application-specific integration; fast regulation, certified protection, and the choice of optimization algorithm remain outside the framework.

The paper makes four contributions:

- 1) a **capability-based industrial execution architecture** connecting agent-team identity, semantic data-acquisition (DAQ) and write-control (DCW) nodes, heterogeneous protocol drivers, and line–product–recipe–run context, with task membership distinct from physical-write authority;
- 2) a **reusable integration boundary** of self-describing node and driver contracts across five protocol families, versioned production objects, plugin extension points, and a typed software development kit (SDK), into which the evaluated reference adapter projects scenario blueprints while process models and policies remain external;
- 3) an **evidence-preserving governed intervention workflow** separating admission, optional approval, actuation, readback, observation, the caller’s verdict, compensation, and backstop actions, so a completed task never stands in for a verified process outcome; and
- 4) a **layered evaluation** exercising one governed path through a mechanism ablation, a deterministic controller, agent-team missions, and concurrent process missions, with task state and objective attainment reported separately throughout.

Scope is stated once here and revisited in Section VI: real software protocol stacks execute against author-implemented dynamic simulators, scripted policies drive the governed tools for

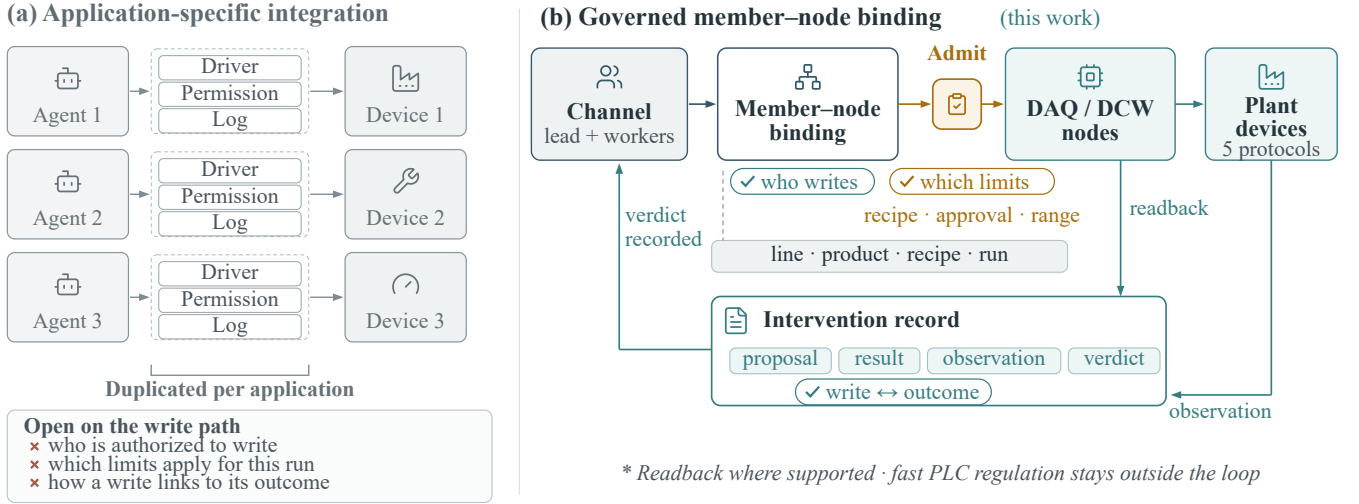


Fig. 1. Conceptual supervisory integration: (a) application-specific links duplicating drivers, permissions, and logging per application; (b) governed member-node binding in AGENTWORKSHOP, where requests pass recipe, approval, and range checks and intervention records return readback, observation, and verdict. Patterns are compared, not optimization performance; PLC regulation remains outside the agent-team loop.

repeatability, and fast regulation, certified protection, physical-plant transfer, and optimizer ranking are outside the claims made here.

II. RELATED WORK

A. Industrial Agents and Closed-Loop Applications

Agent-based manufacturing has long connected organizational roles to industrial capabilities [7]. Recent LLM-oriented work spans industrial knowledge graphs [8], diagnosis [9], PLC generation and verification [3, 4], digital-twin configuration [10], OPC UA machine control [6], and simulated fault handling [5, 11], while InstructMPC combines language-derived objectives with model-predictive control [12]. These studies motivate agent-facing industrial tools; the shared execution layer carrying authority and evidence across scenarios—our focus—is complementary rather than a new controller or reasoning method.

B. Orchestration, Industrial Models, and Governance

AutoGen, LangGraph, and CrewAI coordinate tasks and tools [13–15], and the Model Context Protocol (MCP) and Agent-to-Agent (A2A) specifications standardize tool and inter-agent interfaces [16, 17]. These frameworks coordinate reasoning and tool invocation, but the tool they expose is usually a bare function signature carrying no line, product, or recipe context, with no record separating admission from execution. ISA-88, ISA-95, ISO 23247, and IEC 63278 supply complementary models for recipes, operations, and digital twins [18–21], and AGENTWORKSHOP’s versioned recipes and run dispatch instantiate a reduced subset of that stack: master-recipe-style versions over a line-product-run resource model, without batch procedure models or a full equipment hierarchy. The governed life cycle itself—bounded proposals, post-write holds, per-run journals—transposes the disciplines of run-to-run control [22] onto a supervisory write path; the increment here is interchangeable strategies over

heterogeneous protocols with evidence that survives strategy replacement. Industrial Internet of Things (IIoT) and shop-floor agent architectures likewise emphasize contextualized integration [23], and the nearest neighbors [24, 25] target control decoupling and task decomposition. A recent line instead enforces runtime guarantees on LLM agents—judge and guard agents, capability-based information flow, rule-based enforcement [26–28]; these gate software tool calls; none owns a physical write path with device readback and a write journal; and their policies could compile onto the enforcement substrate contributed here, whose differentiator is production-scoped authority and evidence.

Permissions, range checks, approvals, and audit trails have DCS/SCADA/MES and IEC 62443 counterparts [29], while formal shielding, control-barrier functions, and simplex run-time assurance offer stronger specified properties [30–32]. AGENTWORKSHOP instead implements source-dependent checks and a rule-based backstop and makes no formal protection guarantee: the backstop (Section IV-C) is a rule-based recovery action in the spirit of ISA-18.2 and EEMUA-191 [33, 34] rather than a run-time assurance bound. Human oversight likewise requires more than an approval dialog [35, 36]. Agent benchmarks [37–39], process-monitoring datasets [40, 41], and prompt-injection studies [42, 43] inform adjacent evaluations; ours separates source checks, protocol execution, mechanism ablation, controller trials, and multi-scenario missions.

III. SYSTEM ARCHITECTURE

AGENTWORKSHOP links team coordination to plant interaction through industrial nodes and production context (Fig. 2). Channels coordinate deployed members and tasks; controller services mediate node operations; production objects identify the line, product, recipe, and run. Five planes separate the concerns: a *team* plane decides what to investigate and propose, an *integration* plane presents devices as semantic nodes, a *governance* plane decides whether a request is ad-

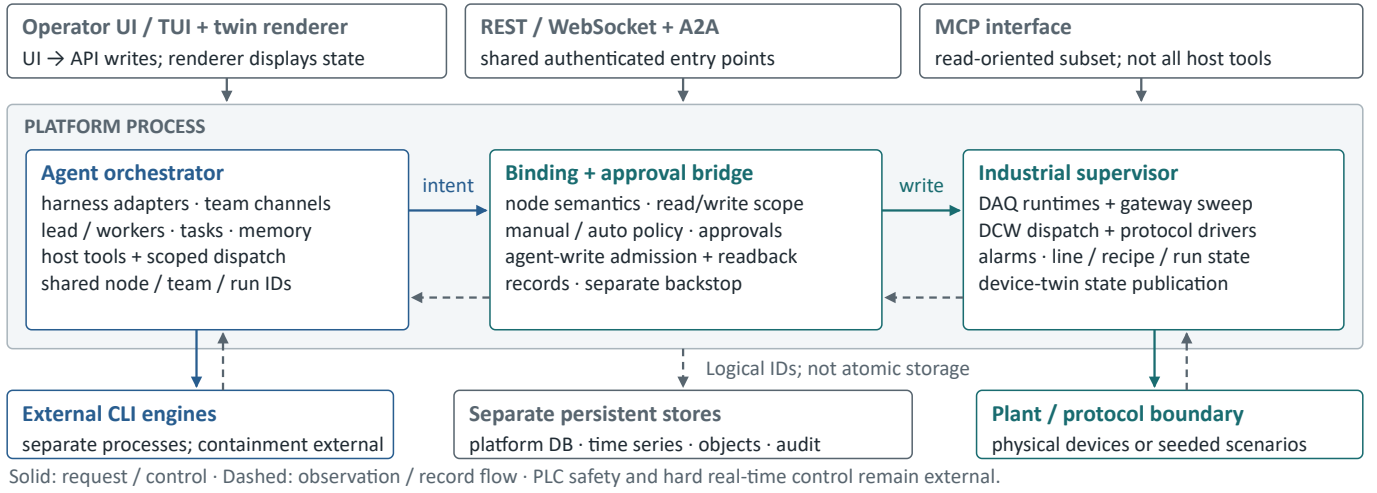


Fig. 2. System architecture and execution boundaries. The platform hosts team routing, binding and approval, industrial supervision, and records. Operator interfaces sit above it; agent engines, data stores, protocol endpoints, and the plant remain separate. Shared identifiers provide attribution but do not make storage and actuation transactional.

missible, an *operations* plane supplies production context and evidence, and an *extensibility* plane admits drivers and services through sanctioned registration points. The decomposition is what makes the framework reusable: scenario knowledge sits in blueprints, signal maps, predicates, and driver configuration, not in team scheduler, approval service, or persistence code.

Each plane draws its own boundary between platform guarantees and scenario inputs: team coordination owns member identity, task state, and a member-written episodic memory retrieved by hybrid search; industrial integration owns protocol acceptance, readback, freshness, and transport errors, its node stream feeding both the agent and a live digital-twin view; governance owns admission, approval, actuation, write readback, observation, verdict, compensation, and rollback; production operations attaches line, product, recipe, and run context while PLC protection stays independent; and extensibility admits plugins, drivers, agent tools, panels, and SDK clients through sanctioned services. In every plane the common execution path is unchanged: what a scenario supplies is data and strategy.

A. Supervisory Core and Nodes

The Node.js core hosts channel routing, industrial services, and protocol adapters. A channel contains a lead, workers, tasks, messages, and retrievable memory, deployed through WebSocket, MCP, A2A, REST, CLI, or TUI entry points; engine adapters preserve member and node identity across runtimes, and channel membership alone confers no write authority.

DAQ nodes convert device readings to engineering quantities; DCW nodes translate engineering setpoints into driver commands and, where supported, readback. Per-node runtimes manage timing; gateway services supply connection state, recipe context, and write bookkeeping. The built-in semantic surface covers Modbus TCP, Modbus RTU-over-TCP, OPC UA, MQTT, and HTTP/REST as both client and server endpoints, plus S7 for acquisition only. The deployment reaches author-implemented dynamic simulators through the same node and driver contracts a plant would use; they are

deterministic per seed but not validated digital twins. The first reuse boundary is declarative: a scenario declares signals, units, scaling, precision, and driver configuration, and the team loop is unchanged; fast regulation and independent protection stay in plant controllers.

Driver contracts are deliberately narrow: a driver declares how a semantic quantity maps to its transport payload, whether the endpoint is writable, and whether a readback is available, while the controller resolves member authority and active limits. Connection, transport, and protocol errors attach to the request rather than one success flag, and register addresses and byte order appear only here, so a new fieldbus family leaves the loop untouched.

Acquisition is supervisory rather than hard real-time: each DAQ node runs its own cadence under a rotating sweep, so a slow endpoint delays only its own samples. The verdict stage queries the timestamped, quality-stamped history rather than issuing a blocking read, which is why a windowed average can lag the device by one cadence.

B. Bindings, Context, and Reuse Boundaries

A member-node binding grants access to a sanctioned tool on a specific node and selects automatic or approval-mediated execution. Line permissions (none, read-only, operate) authorize binding creation and mutation; runtime invocation is authorized by the persisted capability, not channel membership. Intervals are re-checked every request instead of cached, so revoking a binding or switching the recipe takes effect on the next request. The same path serves a console, a harness, and an agent engine, because authority does not depend on the client; only the evidence record differs.

Production objects are versioned rather than overwritten: a recipe is dispatched against a known-good version, reverted non-destructively, and compared with the values actually applied, and an intervention is read against the definition that was active when it was admitted. Plugins register hooks, routes, agent tools, drivers, and panels only through sanctioned services, so an added driver inherits admission, readback, and

evidence capture. A BOPET worker proposes casting-speed, fast-roll, or rail-width changes through bound DCW nodes, and the same mission pattern serves the other three scenarios with only blueprint, guards, and policy differing (Section V-F).

The second reuse boundary is a reference scenario-onboarding adapter (Section III-C); the SDK exposes the same surface to ordinary programs, and runtime settings are hot-reloadable. The stores share identifiers rather than a transaction domain: platform state uses SQLite and files, and measurement and object backends use TimescaleDB and MinIO with local fallbacks. A device may accept a write even if bookkeeping fails, and records lack a run identifier, so time-range queries can cross runs unless bounded.

C. Reference Scenario-Onboarding Contract

The reference integration follows a bounded onboarding contract: the engineer declares a blueprint of devices, signals, units, roles, setpoints, and plant-model parameters; a differential “ensure” pass creates missing devices, repairs drift, verifies connectivity, loads the model without disturbing other lines, projects the blueprint into semantic nodes and a recipe, and tags the line for idempotent reuse; a channel is built from the common team template with the required bindings; and a mission supplies objective, guards, response windows, and policy. The adapter demonstrates reuse of platform services, not configuration-only commissioning of physical plants.

D. Governed Optimization Task Life Cycle

A governed optimization task is a life cycle rather than a single tool call, and AGENTWORKSHOP exposes each transition separately. The task enters a channel carrying goal, line, product, recipe, run, and task identifiers; a lead member claims it and hands it to a worker holding member–node bindings on the relevant nodes. The worker then repeats one fixed job: read the controlled quantity over a bounded window, propose a setpoint inside the resolved interval, invoke the governed write, wait for the response, and record a verdict (Section V-B instruments it).

Each iteration therefore leaves more than a written value: the write opens an optimization record carrying prior and requested values, the admitting interval, the acting member, and timestamps; readback, where supported, closes the command path; the parameter journal attributes the change to its source; and attainment is computed against the scenario predicate while the task closes with a workflow terminal state. This bounds what “automatic optimization” means here: a controller, a scripted policy, a conventional optimizer, or an agent engine may occupy the strategy slot, while authority, evidence, and closure stay on the platform—and a mission may reach its terminal state while missing the process objective, because the two are never merged.

IV. GOVERNED EXECUTION MECHANISMS

A supervisory intervention must preserve distinctions that are often collapsed in an agent tool call—permission to propose, admission of a value, driver acceptance, readback,

process response, a caller’s verdict, and compensation—and AGENTWORKSHOP exposes these as separate, distinguishable evidence states. Fig. 3 follows one such intervention through the implemented runtime. In it, every path that reaches the plant passes through the same governed write—the four agents are interchangeable participants, not control primitives—and the goal marker terminates the workflow, which is why attainment is recorded separately from task state throughout. The subsections below follow that path—admission (Section IV-A), write and readback (Section IV-B), observation, verdict, and compensation (Section IV-C), approval (Section IV-D), and the stable plane’s reuse boundary (Section IV-E)—and Fig. 4 shows the path on the running operator surfaces.

A. Task, Observation, and Admission

The strategy receives a task and goal in production context and decides what to propose and when to stop; the platform governs execution. Decision variables are semantic node parameters—pressure, dosing rate, line speed, rail width—in engineering units. Authority belongs to deployed member–node bindings, not the channel, prompt, or task template.

Before a proposal, `daq_query` obtains a baseline or response window and reports freshness. For an ordinary agent setpoint request on node n , the implemented admission interval is

$$\mathcal{I}_n(t) = \mathcal{H}_n \cap \mathcal{B}_n \cap \mathcal{P}_n(t) \cap \mathcal{R}_n(t), \quad (1)$$

where $\mathcal{H}_n$ is the hard engineering range and the remaining terms the baseline, active-product, and active-recipe limits; missing optional limits add no restriction. The intersection constrains submitted values—it is not a learned feasibility region or a physical safety certificate—and availability, binding state, and approval policy are checked separately. The intersection bounds magnitude, not intent: in-window but adverse sequences surface at judgment and compensation (Section V-F reports two).

B. Semantic Write and Source-Specific Routes

A semantic card exposes quantity, unit, precision, range, value, freshness, and recipe context. The controller resolves the binding and the limits; the driver then serializes the command. Readback, where supported, closes the command path, not the process-response path.

After a successful agent or manual write the node installs a post-write hold window (`writeLockSeconds`, default 30 s); a further agent-origin request inside it receives HTTP 429 `WRITE_FREQUENT`. The lock is installed only after driver success, recipe-dispatch and rollback writes are exempt, and zero disables it. `AW_BENCH_MODE=1` admits harness-declared ablation arms on the manual route that relax the soft window (the Stage-1 gate of Fig. 3), the readback check, or the hold—the hard engineering range is never bypassable. The hold is per node and agent-origin only: it guards a loop whose response window may be comparable to the sampling cadence, not a rate limit on the plant.

Four governed routes—agent tools, manual writes, recipe dispatch, and rollback compensation—reach the same driver

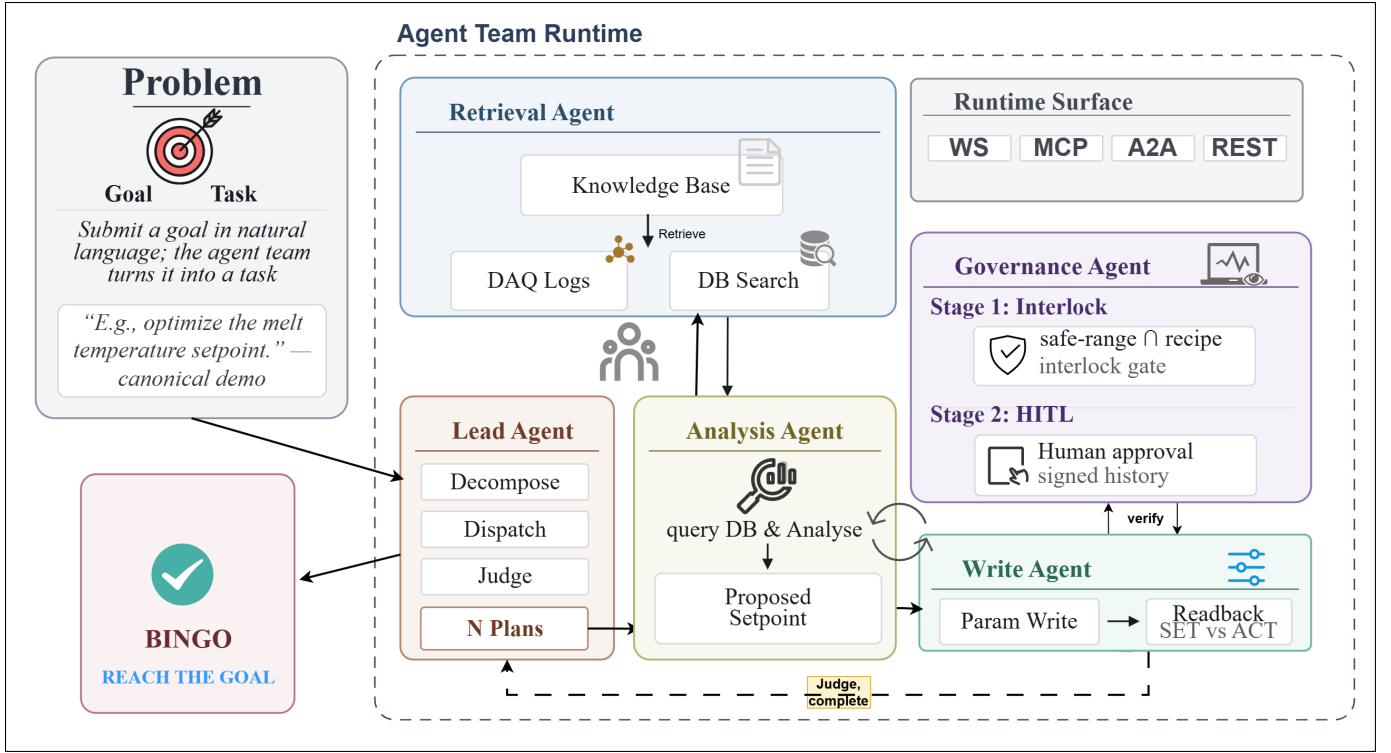


Fig. 3. Implemented agent-team runtime. A natural-language goal enters as a task (left); the lead agent decomposes it and dispatches one of N plans; the retrieval and analysis agents produce the proposed setpoint; the write agent performs the write and reads the value actually present back (SET versus ACT). The governance agent is a deterministic two-stage gate: Stage 1 intersects the safe range with the active recipe window (drawn simplified; the implemented admission intersects hard, baseline, product, and recipe limits, Eq. (1)), and Stage 2 inserts the human approval wait on manual bindings, both journaled with actor attribution. A judge/complete verdict returns to the lead agent, and the goal is marked reached only along this governed path (entered over WebSocket, MCP, A2A, or REST)—a reached goal ends the governed path, not evidence that the process objective was met.

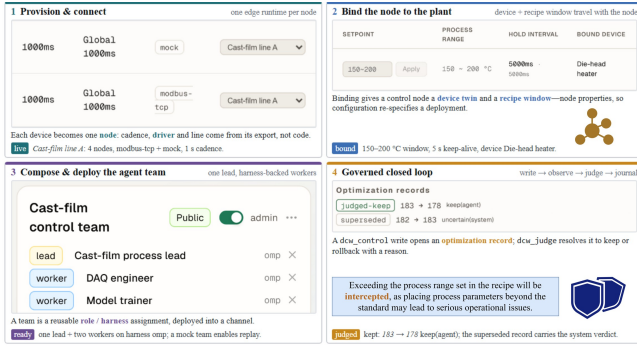


Fig. 4. The implemented path on the running operator surfaces: (1) provisioned nodes carry cadence, driver, and line assignment; (2) a binding attaches the node to its device under the active range and hold interval; (3) a channel deploys one lead and two workers (Fig. 3's four roles are logical and map onto lead and worker instances); (4) a governed closed loop opens an optimization record and stores a verdict. Running UI/API surfaces, not benchmark outcomes.

through one admission and journaling entry point, the hard range included; recipe dispatch adds run context, its values validated against the recipe window when the version was saved. Hold heartbeats are the one non-governed re-dispatch—periodic rewrites of an already-admitted value, sharing runtime write exclusion without re-passing admission or conferring member authority. Line start installs active context before recipe parameters dispatch individually, and line stop clears context and closes retained open records. These multi-step operations are not atomic with device state.

C. Observation, Judgment, and Compensation

An accepted write opens an optimization record; the verdict tool `dcw_judge` records keep, rollback, or uncertain plus a reason; it performs no actuation and does not independently validate the evidence, so a scripted keep can coexist with a missed objective. The verdict is stored with the record it closes and the response window actually queried, so a later reader sees the evidence in front of the caller. Verdicts are attributable—the actor and timestamp travel with them; corrections amend the stored reason visibly rather than replacing the verdict, and compensation opens a new record that references the one it reverses.

Compensation is a new, separately admitted write that may fail or partially complete; its dispatch is read-back verified, and a rollback record settles to rolled back only once the compensating value is confirmed at the device, while recipe rollback restores a definition, not physical PLC values. These separations keep command status, record status, and process recovery independently inspectable (Fig. 4).

A stored-policy backstop evaluates open records: it waits at least 120 s, checks at most every 30 s, and triggers after three returned points or averages leave the window; recovery executes only while fewer than two retained rollback records stand on the node—a deliberate de-arm point, past which the backstop still judges and records and defers actuation to the record's approval policy. The cadence is neither a deadline nor a formal run-time assurance bound.

D. Human Approval Boundary

Manual bindings insert a timed approval wait before an agent setpoint or compensation request; automatic bindings omit the wait but keep the checks. The pending request shows member, node, value, interval, and expiry; approval reserves neither interval nor device state, so binding and admission are re-checked at execution. The wait fails closed: an expired request resolves as refusal and does not actuate, with expiry and refusal recorded; duplicates are suppressed. Selecting the approval-free mode is itself a user-side, line-permission configuration action—agent identities hold no binding-administration capability—so an agent cannot reclassify its own writes as approval-free.

The decision endpoint authenticates a user but does not enforce a line-scoped approver role, and reads, verdict recording, and some configuration changes do not use physical-write approval: this mechanism demonstrates workflow mediation, not segregation of duties or certified human supervision. A recipe switch between approval and execution re-admits the value against the new window rather than re-presenting it to the approver; the value must still satisfy the changed interval or be rejected.

E. Design Rationale and Reuse Boundary

The strategy engine is replaceable rather than trusted as a control primitive. The stable plane keeps bindings, semantic I/O, evidence, approval, compensation, and backstop handling, while a scenario supplies blueprint, limits, objective, response window, and policy, confining scenario-specific work without claiming physical portability. Two rules enforce that boundary: scenario code may declare and configure but never execute plant I/O, so every write still passes admission, driver dispatch, and evidence capture; and the platform never infers a process objective, so records can be re-evaluated against a revised predicate without rewriting history.

V. EXPERIMENTAL EVALUATION

The evaluation asks whether one governed supervisory path can be exercised, inspected, and repeated across heterogeneous industrial jobs, not whether one optimizer beats another. Four questions organize the evidence: protocol closure and governance attribution (Q1), the AgentTeam workflow (the lead-worker team runtime of Fig. 3) on one node and on a multi-node line (Q2), the deterministic cast-film loop (Q3), and attainment for four process missions (Q4), each instantiating the path of Fig. 4.

A. Testbed, Evidence Snapshot, and Scope

The snapshot is integrated run 20260921184204-101c (seed 42, commit e692df1, harness 9f2756dce4a4cbb3, completed September 21, 2026, 19:08 UTC), archived with raw reports and mission logs; the ablation companion is 20260921181234-e1lite. Scripted external policies, rather than language models, call the governed tools under a deployed member identity, so results are repeatable. Two live agent-engine turns check that the same slot accepts

an engine; their archived execution-trace headers record the harness (omp), the provider, and the model (glm-5.3-flash). The standing single-node preset is a cast-film extrusion line tuned through the die-head heater setpoint, with objective J aggregating five observed quantities (larger is better); each scenario contributes one trajectory from its seed-42 initial state.

B. Test Procedure and System-Level Profile

The procedure is a 20-phase integrated suite that scores 83/83 checks with no warnings or failures—an author-defined regression summary, not an external certification. Each phase probes one claim, from preset loading against the offline optimum ($J^* = 89.894$) and per-protocol interception, through the record, recipe, approval, and parameter-layer drills and the controller benchmark, to the second preset, the backstop drill, and the concurrent campaign (Section V-F).

Figure 5 reports where the time goes: Panel (a) shows each phase's wall time (log scale), and three tiers appear. Deterministic governance steps—admission, binding checks, record writes, recipe lifecycle—complete in milliseconds to seconds, so the platform's per-intervention cost is validation logic; the long tail sits in the replaceable strategy slot and the observation cadences: the live agent-engine turns (P5, P5b) take 156.3 s and 637.3 s, and the backstop drill (P8b) spans 134.7 s, within which the system-judged rollback fires at 130.2 s. The framework stays thin, and the expensive tiers are exactly the ones a deployment would swap. Panel (b) is the quality tier: the closed-loop objective after two writes, normalized by J^* ; all three seeds land within 3.5 percentage points of the oracle and within one point of one another (Section V-E).

C. Q1: Protocol Closure and Governance Attribution

On the five protocol lines the suite stores 55 DAQ samples through real drivers, including a Modbus RTU-over-TCP satellite; governed writes are accepted and read back on Modbus TCP, OPC UA, MQTT, and HTTP with per-line medians of 47.8 ms, 19.8 ms, 19.3 ms, and 32.9 ms, pooling to 29.954 ms. Because one admission, record, and verdict path serves every request, the residual spread is consistent with transport differences. Each writable line rejects 6/6 configured out-of-window probes while accepting its legal write, so all 24 probes are intercepted; a dedicated window-interlock drill (the Stage-1 gate of Fig. 3) adds decision-boundary probes at the window edge and just outside it, and exercises both admission branches, line running and line stopped—the interlock discriminates rather than merely blocks.

Two drills then separate truth paths that one success flag would collapse: a frozen-measurement drill accepts a write and a consistent readback while the measurement stays out of window, and the backstop records a system-judged rollback 130.2 s later—readback closes the command path, the backstop closes the record, and neither is a settling time. A second preset with different signals and dynamics is then commissioned by configuration alone—three writable lines and two satellite nodes with zero framework code changes. Across both presets

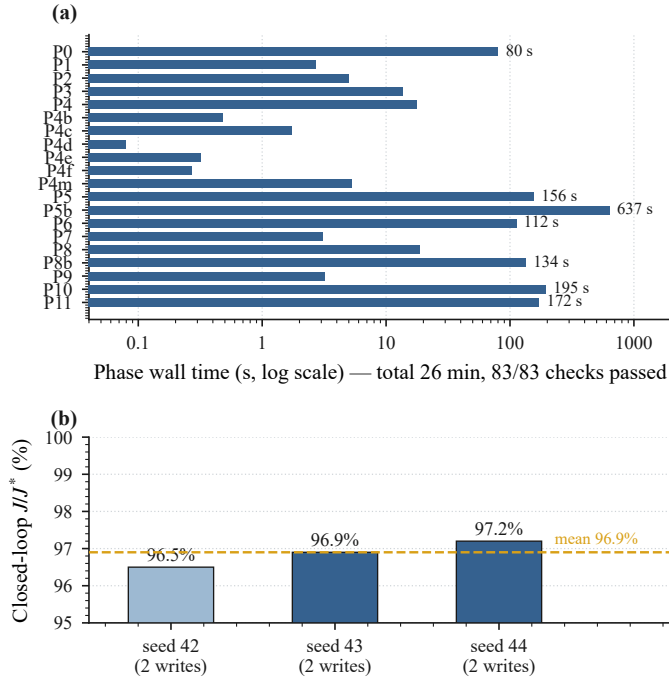


Fig. 5. Run 20260921184204-101cat system level. (a) Phase wall time (log scale); bar labels are harness phase identifiers. (b) Closed-loop objective J after two governed writes as a fraction of J^* , per seed, with no write rejected. Panel (a) is environment telemetry, not a service-level bound.

TABLE I
GOVERNANCE ABLATION ON COMPANION RUN
20260921181234-ELITE (THREE DETERMINISTIC REPETITIONS WITH
IDENTICAL COUNTS; 18 PROBES PER ARM: 12 HARD-RANGE AND 6
WINDOW-CLASS).

Arm	Rejected	Executed breaches	False blocks
Full governance	18/18	0	0
No window interlock	12/18	6	0
No readback check	18/18	0	0
Both relaxed	12/18	6	0

33 configured probes are rejected on seven writable lines, the same admission code discriminating on signals it had never seen.

The parameter layer passes 4/4 checks and makes the interval geometry of Section IV-A observable: a semantic surface without register leakage, and a four-layer chain (node [50, 200] rpm, parameter [143.7, 156.3] rpm, product [146.22, 153.78] rpm, recipe [141, 159] rpm) narrowing to one admitted interval, each rejection attributable to its excluding layer. A manual binding instead suspends a request visibly and unblocks it on approval, after which the write takes effect at the device (automated round-trip latency 461 ms): workflow mediation, not physics.

Table I attributes the rejection rather than assuming it, on a pipeline-tier companion fixture kept separate from the five protocol lines; counts are identical across the three deterministic repetitions, and the per-request latencies archived for every arm are indistinguishable at the archived resolution—the validation logic adds no measurable overhead on this fixture. Full governance and the no-readback arm reject 18/18 probes, whereas removing the window interlock, alone or with the

readback check, lets the six window-class probes execute and be journaled. Two readings follow. First, prevention is the window interlock’s job: the readback check contributes nothing to interception—it closes the command path after execution—yet is retained because it makes a written-but-unconfirmed command visible. Second, attribution survives removal: the six executed breaches still appear in the journal, so a relaxed configuration degrades from prevention to detection. The hard range is structurally unbypassable and the soft window is its removable part; no arm records a false block.

D. Q2: The AgentTeam Optimization Workflow and Its Results

The workflow ran end to end on the cast-film line and then on a larger BOPET line (Section III-D); a scripted policy rather than a language model issued the writes in both missions; the two live engine turns follow at the end of this subsection. On the single-node mission one write sufficed: the die-head heater setpoint moved from 200 to 204.704 °C, the observation settled at 204.70 °C against 204.704 ± 0.75 °C, the journal attributed the change to the agent identity, and the task closed COMPLETED.

The BOPET mission is the multi-node case: nine devices and 49 signals (30 writable, 19 readable), all created by the *ensure* pass, whose driver tests passed on all nine endpoints. Thickness started at 28.10 μm against 25.0 ± 0.7 μm. Three governed writes on three distinct nodes—casting speed 32 → 33.8 m/min, fast-roll speed 118 → 120.3 m/min, and rail width 3000 → 3035 mm—brought it to 25.66 μm. Each write opened a record, received *keep*, and left an agent-attributed journal entry, while melt temperature stayed near 291 °C and the parent task reached COMPLETED. In both missions the governed-write count did not scale with node count: each write traversed the same admission, record, and verdict steps regardless of line size.

Two live engine turns show that the strategy slot also accepts a language model. In the bounded turn the agent read the node’s semantic card, sampled a baseline through *daq_query*, wrote 197.48 °C, waited, resampled, and closed the record with *keep*, finishing in 156.3 s and storing one lesson: the acquisition-limit flag can lag the live value. The goal-driven turn took 637.3 s. It queried the journal and its own memory, found neither, and measured 53.79 μm at 150 rpm against a self-selected target of 52.0 ± 0.8 μm; the governed path, not a predicate, is what constrains this turn. From mass conservation ($h \propto N/v$: thickness to screw speed over line speed) it inferred that screw speed should fall by about 5 rpm, applied the change in steps, judged each write on measured evidence, refined its gain estimate from 0.22 to 0.36 μm per rpm, and settled thickness at 51.7 μm. It also saved the recipe—authorship attributed and audit-logged, activation role-gated.

E. Q3: The Deterministic Closed Loop

Seeds 42–44 each issue two governed writes with no rejection, raising the cast-film objective J (larger is better) from $J_0 \approx 67.2$ to $J/J^* = 0.965, 0.969$, and 0.972 on the three seeds, against the simulator’s offline reference $J^* = 89.894$

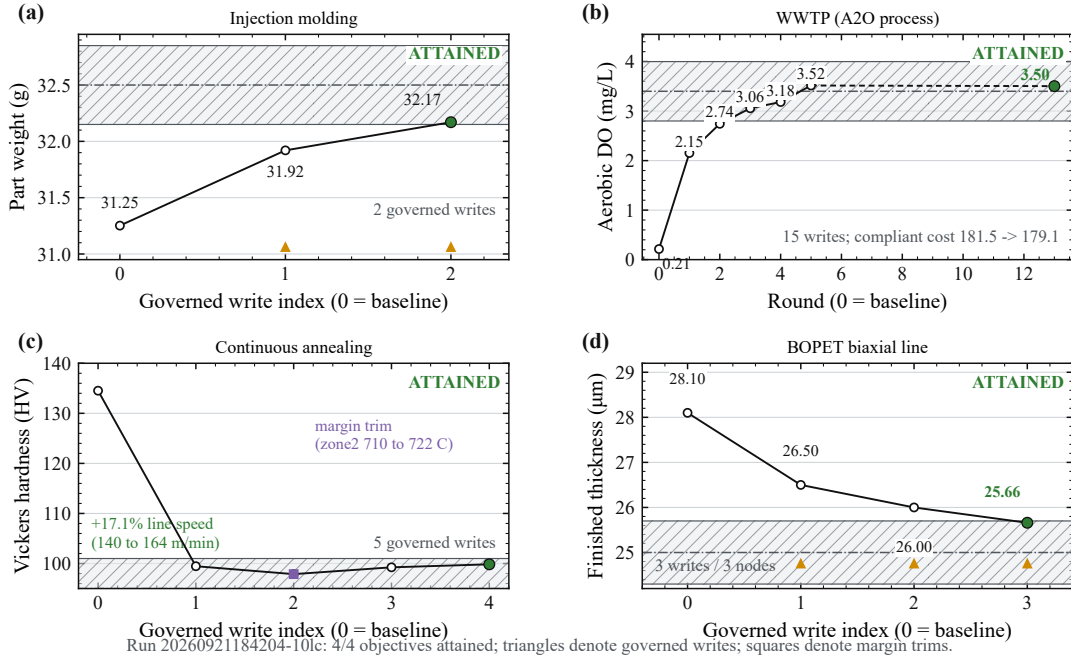


Fig. 6. Archived trajectories of the four missions in run 20260921184204-101c. Hatched bands are the objective windows, the dash-dotted line the target, and filled terminal markers the archived observations satisfying the predicate. Panel (a) marks governed-write rounds with triangles, Panels (c) and (d) index points by governed write, and Panel (b) tracks dissolved oxygen through the mission's stages (its round axis includes the 15 governed writes) with write count and cost index annotated; the square in (c) marks the zone-2 margin trim. All four terminal markers lie inside their objective window. Curves are simulator observations under scripted policies, not physical measurements.

obtained by exhaustive grid search over its steady-state map. The residual gap to unity is expected: J^* is an oracle with full model knowledge, which an online policy seeing only process observations cannot in general match. What the benchmark establishes is narrower: the governed path passes a conventional controller's writes through unmodified—admissible, journaled, and read back—and no ungoverned performance arm was run, so the path's transparency is shown for admission and evidence handling rather than measured as a performance delta.

F. Q4: Four Process Missions

The campaign runs four lines concurrently, each provisioned from a blueprint by the *ensure* pass and given one channel; all four attain their archived predicates, evaluated from subsequent observations rather than from the task state. The controlled quantity moved from 31.25 to 32.17 g, 0.21 to 3.50 mg/L, 134.52 to 100.08 HV, and 28.10 to 25.66 μm , with mission clocks of 26.48 s, 169.27 s, and 102.76 s (the BOPET clock was not retained). Figure 6 archives the trajectories: the curve is the observed controlled variable, the hatched band its window, and the filled terminal marker the predicate-satisfying observation. Each mission iterates the same loop—read the window, write inside the admitted interval, observe the response, judge—so one governed loop produces all four trajectories.

Panel (a): injection molding is the minimal case—two writes raise the hold pressure from 45 to 58.2 bar and the hold time from 6 to 10.64 s, and the policy stops at 32.17 g inside 32.5 ± 0.35 g in 26.48 s, with the flash and sink guards inside their limits.

Panel (b): wastewater treatment, the longest mission, has two stages. Aeration, caustic, and coagulant dosing bring

dissolved oxygen from 0.21 into 3.4 ± 0.6 mg/L under the effluent guards on COD, ammonia, total phosphorus, pH, and turbidity; the policy then lowers blower frequency and caustic dose, ending after 15 writes at 3.50 mg/L in 169.27 s with the simulator-internal compliance-cost index (Fig. 6b) down from 181.5 to 179.1. Two blower steps are the most informative events in the campaign: both writes were admissible and journaled, the post-write observation lost compliance margin, and the verdict stage reverted them—so rejection of a bad proposal lives in the evidence, not in the admission gate.

Panel (c): annealing starts far outside the hardness window (134.52 HV at 140 m/min). Raising zone-3 temperature to 850 °C brings hardness to 99.46 HV; one zone-2 trim and two line-speed steps then hold 95 ± 6 HV while raising line speed by 17.1%, and a further attempt at 176 m/min is reverted for leaving the margin. All 5 governed writes are marked on the panel's baseline-indexed axis. The guard here is multi-objective—hardness and throughput must hold together—and the predicate holds in 102.76 s only because the margin-violating speed was undone, on record.

Panel (d): BOPET is the nine-device mission of Section V-D executed under the campaign harness, moving thickness from 28.10 to 25.66 μm inside 25.0 ± 0.7 μm . In this mission completion and the predicate agree, yet they remain separate records, so a later missed objective would still be visible.

G. Cross-Scenario Reading

Read together, the four missions bound what the framework contributes. Platform-side behavior is invariant—admission, driver I/O, records, verdicts, closure—while the write economy varies by more than a factor of seven (2, 15, 5, and 3 writes

across the four lines) and the mission clock by more than a factor of six (26.48 s to 169.27 s over the three retained clocks). Both ranges are produced by construction in the scenario-specific slot: blueprint, guards, and policy. The three reversions (two blower steps in WWTP, one speed step in annealing) show caller-side verdicts doing real work with the platform preserving the distinction; that all four terminal markers lie inside their bands is an observed outcome of this run, not a structural property. Margins are narrow but on record: three terminals sit within 1% of a band edge (0.02 g, 0.04 μ m, 0.92 HV to the nearest bound), and the compliance-cost index ends 1.3% lower.

VI. DISCUSSION AND LIMITATIONS

A. Industrial-Informatics Interpretation

The principal result is that one supervisory path carried four different jobs while keeping command closure, the submitted verdict, task state, and process attainment as separate records. In run 20260921184204-101c, all four scenario predicates hold, and the governed closed loop reaches J/J^* between 0.965 and 0.972 (mean 0.969): a range rejection points to a permission or window change, a driver or readback failure to an execution problem, and an accepted write followed by an adverse response to the need for revision or compensation—three distinctions a single success flag would merge.

B. Validity and Reproducibility

The snapshot fixes the run identifier, seed, commit, harness fingerprint, and raw artifacts, so the reported counts can be re-derived rather than trusted. Four qualifications bound what they support. First, the evidence is simulation-tier: the simulators are deterministic per seed but are not validated digital twins, and no physical-device campaign, operator study, Tennessee Eastman Process (TEP)/Secure Water Treatment (SWaT) replay, or external-framework comparison is reported. Second, the controller optimum and the scenario predicates are internal oracles, and the ablation has three deterministic repetitions, so it supports mechanism attribution rather than statistical inference; latencies are environment observations, not service-level bounds. Third, initial state, asynchronous sampling, response windows, and simulator timing still influence deterministic policies. Fourth, several capabilities described in Section III are implemented but not exercised here, including memory retrieval quality, plugin hot-reload, alarm-chain timing, and engine interchange; the memory call made by the live turns returns no stored entry, so the memory layer is exercised rather than validated. The four process models broaden structural variety within those bounds but do not establish transfer to actual equipment.

C. Safety Positioning and Human Oversight

The objection that a language-model team should not “take over” production applies to a design AGENTWORKSHOP does not propose: fast regulation, hardware interlocks, and safety functions remain in the PLC, and Section IV-D already bounds what an agent can change. What the framework adds

is not a protection guarantee but a bounded blast radius: prompts, retrieved documents, and member-written memory are untrusted text rather than authority [42, 43], so a mistaken or manipulated proposer is confined to per-binding scope and journaled, attributable actions. Human oversight rests on the same states: binding scope, admission limits, readback, and the backstop operate whether or not a human approves, which bounds the damage of approval fatigue and automation bias [35, 36, 44, 45]. Approval is a revocable per-binding policy, and because admission is re-checked at execution, a stale approval cannot carry an out-of-window value; a context change between approval and execution is caught by that re-admission rather than by re-consulting the approver. Oversight consumes the evidence record rather than an alarm stream.

D. Deployment Boundaries

Execution and recovery are non-atomic. Bookkeeping may fail after actuation, the post-write hold guards write frequency rather than stability, readback agreement need not imply process recovery, and multi-node compensation may partially succeed, while within-range oscillation, storage failure, clock error, and worst-case recovery time remain unevaluated.

For any production deployment, bindings govern sanctioned tools, not arbitrary I/O from a privileged engine, so least privilege, endpoint security, and IT/OT separation remain necessary [29, 46], and independent hardware interlocks and any required IEC 61511 protection must stay in place: AGENTWORKSHOP is neither a safety instrumented function nor a substitute for plant protection [47]. The approval wait already fails closed on expiry, but the endpoint still lacks line-scoped approver authorization, so a production deployment would additionally need segregation of duties and alarm-path failure tests [34].

VII. CONCLUSION

AGENTWORKSHOP contributes a governed supervisory execution architecture rather than a new process optimizer: a stable path linking agent-team identities, semantic nodes, production context, governed writes, readback, observations, verdicts, and compensation, exercised by a 20-phase suite, agent-team missions, a three-seed loop, and four process missions. In the reference run all 83 checks pass, every configured full-governance probe is rejected, and all four mission predicates hold, with command acceptance, readback, the submitted verdict, and task completion kept distinct from those predicates. Physical-plant deployment, human-operator studies, and external-framework comparison remain open; until they are reported, claims of optimization superiority, autonomous control, and safety assurance stay unsupported.

DATA AVAILABILITY AND DISCLOSURE

Artifacts are archived in `bench/results/`: the main run in `bench/results/20260921184204-101c`, the ablation companion in `bench/results/20260921181234-e11ite`. Reproduction also needs the simulator source and reset state, and the snapshot lacks an immutable simulator-tree fingerprint. AI assistants supported editing and evidence checking under author direction; the authors remain responsible for all claims and data.

REFERENCES

- [1] M. Raza, Z. Jahangir, M. B. Riaz, M. J. Saeed, and M. A. Sattar, "Industrial applications of large language models," *Sci. Rep.*, vol. 15, 2025, art. no. 13755.
- [2] S. Colabianchi, F. Costantino, and N. Sabetta, "Assessment of a large language model based digital intelligent assistant in assembly manufacturing," *Comput. Ind.*, vol. 162, 2024, art. no. 104129.
- [3] Z. Liu, R. Zeng, D. Wang, G. Peng, X. Liu, Q. Liu, P. Liu, W. Wang, and J. Wang, "Agents4PLC: Automating closed-loop PLC code generation and verification in industrial control systems using LLM-based agents," *IEEE Trans. Softw. Eng.*, vol. 52, no. 5, pp. 1672–1687, 2026.
- [4] M. Fakihi, R. Dharmaji, Y. Moghaddas, G. Quiros, O. Ogundare, and M. A. Al Faruque, "LLM4PLC: Harnessing large language models for verifiable programming of PLCs in industrial control systems," in *Proc. 46th Int. Conf. on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2024, pp. 192–203.
- [5] J. Vyas, M. S. Gill, A. Markaj, F. Gehlhoff, and M. Mercangöz, "From detection to action: Using LLM agents for fault-tolerant control," *arXiv preprint arXiv:2606.28011*, 2026.
- [6] B. Hofmann, N. Piechulek, S. Kreitlein, J. Franke, and P. Bruendl, "Beyond touch-based human-machine interface: Control your machines in natural language by utilizing large language models and OPC UA," *arXiv preprint arXiv:2510.11300*, 2025.
- [7] P. Leitão, "Agent-based distributed manufacturing control: A state-of-the-art survey," *Eng. Appl. Artif. Intell.*, vol. 22, no. 7, pp. 979–991, 2009.
- [8] J. Zhang, H. Wang, Z. Jia, J. Dong, and L. Ren, "CoMA-IGK: LLM-driven multiagent framework for automated construction of industrial knowledge graph," *IEEE Trans. Ind. Inform.*, vol. 22, no. 6, pp. 5553–5564, 2026.
- [9] P. Liu, L. Qian, X. Zhao, and B. Tao, "Joint knowledge graph and large language model for fault diagnosis and its application in aviation assembly," *IEEE Trans. Ind. Inform.*, vol. 20, no. 6, pp. 8160–8169, 2024.
- [10] Y. Xia, D. Dittler, N. Jazdi, H. Chen, and M. Weyrich, "LLM experiments with simulation: Large language model multi-agent system for simulation model parametrization in digital twins," in *Proc. IEEE ETFA*, 2024, pp. 1–4, arXiv preprint arXiv:2405.18092.
- [11] M. S. Gill, J. Vyas, A. Markaj, F. Gehlhoff, and M. Mercangöz, "Leveraging LLM agents and digital twins for fault handling in process plants," in *Proc. IEEE ETFA*, 2025, pp. 1–8, arXiv preprint arXiv:2505.02076.
- [12] R. Wu, J. Ai, and T. Li, "InstructMPC: A human-LLM-in-the-loop framework for context-aware control," in *Proc. 2025 IEEE 64th Conference on Decision and Control (CDC)*, 2025, pp. 172–179.
- [13] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," *arXiv preprint arXiv:2308.08155*, 2023.
- [14] LangChain, Inc., "LangGraph," [Online]. Available: <https://github.com/langchain-ai/langgraph>, 2024, accessed: Sep. 20, 2026.
- [15] CrewAI, Inc., "CrewAI," [Online]. Available: <https://github.com/crewAIInc/crewAI>, 2024, accessed: Sep. 20, 2026.
- [16] Anthropic, "Model context protocol," [Online]. Available: <https://modelcontextprotocol.io>, 2024, accessed: Sep. 20, 2026.
- [17] Linux Foundation, "Agent2Agent (A2A) protocol," [Online]. Available: <https://a2a-protocol.org>, 2025, accessed: Sep. 20, 2026.
- [18] International Society of Automation, "ISA-88.00.01-2010—batch control—part 1: Models and terminology," Standard, 2010.
- [19] —, "ANSI/ISA-95.00.01: Enterprise-control system integration," Standard, 2010.
- [20] International Organization for Standardization, "ISO 23247: Digital twin framework for manufacturing," Standard, 2021.
- [21] International Electrotechnical Commission, "IEC 63278-1:2023: Asset administration shell for industrial applications—part 1: Asset administration shell structure," International Standard, 2023.
- [22] J. Moyné, E. del Castillo, and A. M. Hurwitz, *Run-to-Run Control in Semiconductor Manufacturing*. Boca Raton, FL: CRC Press, 2000.
- [23] A. Gautam, M. R. Aryal, S. Deshpande, S. Padalkar, M. Nikolaenko, M. Tang, and S. Anand, "IIoT-enabled digital twin for legacy and smart factory machines with LLM integration," *J. Manuf. Syst.*, vol. 80, pp. 511–523, 2025.
- [24] A. Biskupovic, A. Villalonga, F. Castaño, R. E. Haber, and F. Núñez, "Agent-based supervision for service-oriented industrial cyber-physical systems," *IEEE Trans. Ind. Inform.*, vol. 21, no. 3, pp. 2719–2728, 2025.
- [25] Z. Zhao, D. Tang, C. Liu, L. Wang, Z. Zhang, H. Zhu, K. Chen, Q. Nie, and Y. Ji, "A large language model-based multi-agent manufacturing system for intelligent shopfloors," *Adv. Eng. Inform.*, vol. 69, 2026, art. no. 103888.
- [26] Z. Xiang *et al.*, "GuardAgent: Safeguard LLM agents by a guard agent via knowledge-enabled reasoning," 2024.
- [27] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, "Defeating prompt injections by design," 2025.
- [28] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents," 2025.
- [29] International Electrotechnical Commission, "IEC 62443-3-3: System security requirements and security levels," Standard, 2013.
- [30] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, "Safe reinforcement learning via shielding," in *Proc. AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018.
- [31] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, "Control barrier function based quadratic programs for safety critical systems," *IEEE Trans. Autom. Control*, vol. 62, no. 8, pp. 3861–3876, 2017.
- [32] L. Sha, "Using simplicity to control complexity," *IEEE Softw.*, vol. 18, no. 4, pp. 20–28, 2001.
- [33] International Society of Automation, "Ansi/isa-18.2: Management of alarm systems for the process industries (iec 62682)," Standard, 2016.
- [34] Engineering Equipment and Materials Users Association, "EEMUA 191: Alarm systems—a guide to design, management and procurement," Guide, Edition 3, 2013.
- [35] R. Parasuraman, T. B. Sheridan, and C. D. Wickens, "A model for types and levels of human interaction with automation," *IEEE Trans. Syst., Man, Cybern., Part A*, vol. 30, no. 3, pp. 286–297, 2000.
- [36] L. Bainbridge, "Ironies of automation," *Automatica*, vol. 19, no. 6, pp. 775–779, 1983.
- [37] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, S. Zhang, X. Deng, A. Zeng, Z. Du, C. Zhang, S. Shen, T. Zhang, Y. Su, H. Sun, M. Huang, Y. Dong, and J. Tang, "AgentBench: Evaluating LLMs as agents," in *Proc. ICLR*, 2024, arXiv preprint arXiv:2308.03688.
- [38] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, " τ -bench: A benchmark for tool-agent-user interaction in real-world domains," *arXiv preprint arXiv:2406.12045*, 2024.
- [39] F. F. Xu, Y. Song, B. Li, Y. Tang, K. Jain, M. Bao, Z. Z. Wang, X. Zhou, Z. Guo, M. Cao, M. Yang, H. Y. Lu, A. Martin, Z. Su, L. Maben, R. Mehta, W. Chi, L. Jang, Y. Xie, S. Zhou, and G. Neubig, "Theagentcompany: Benchmarking LLM agents on consequential real world tasks," *arXiv preprint arXiv:2412.14161*, 2024.
- [40] J. J. Downs and E. F. Vogel, "A plant-wide industrial process control problem," *Comput. Chem. Eng.*, vol. 17, no. 3, pp. 245–255, 1993.
- [41] J. Goh, S. Adepu, K. N. Junejo, and A. Mathur, "A dataset to support research in the design of secure water treatment systems," in *Proc. 11th Int. Conf. Critical Information Infrastructures Security (CRITIS)*, 2017, pp. 88–99.
- [42] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop on Artificial Intelligence and Security*, 2023, pp. 79–90.
- [43] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in *Proc. 33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, 2024, pp. 1831–1847. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity24/presentation/liu-yupei>
- [44] J. D. Lee and K. A. See, "Trust in automation: Designing for appropriate reliance," *Hum. Factors*, vol. 46, no. 1, pp. 50–80, 2004.
- [45] R. Parasuraman and D. H. Manzey, "Complacency and bias in human use of automation: An attentional integration," *Hum. Factors*, vol. 52, no. 3, pp. 381–410, 2010.
- [46] National Institute of Standards and Technology, "NIST SP 800-82 rev. 3: Guide to operational technology (OT) security," Special Publication, 2023.
- [47] International Electrotechnical Commission, "IEC 61511-1:2016: Functional safety—safety instrumented systems for the process industry sector—part 1: Framework, definitions, system, hardware and application programming requirements," International Standard, 2016.